Student Meal Planner

Layered Architecture Research

A Technical Research Report for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

Introduction

Grubs4Scrubs started as a single ASP.NET Core Web API project with everything crammed into one place. Controllers talked directly to the database. SQL queries sat right next to HTTP response logic. It worked, technically. But it was messy, hard to extend, and my coach flagged it as something that needed fixing before the project grew any further.

The question I wanted to answer was straightforward: how should I split my backend into layers, and what does each layer actually do? This matters for Grubs4Scrubs specifically because the app is growing. We've got recipes, users, meal planning, shopping lists, and eventually authentication. If all of that stays in one flat project, every new feature makes the codebase harder to work with. I picked this topic because my coach explicitly asked me to implement a layered architecture, and I wanted to understand why, not just how.

Main Research Question

What is the best way to structure the Grubs4Scrubs ASP.NET Core backend into layers, and what are the benefits of doing so?

Sub-Questions, Methods, and Approach

To answer the main question, I broke it down into four sub-questions. Each one targets a different angle of the same problem. I used methods from the DOT framework to approach them, mostly library research and available product analysis since this is a well-documented topic in the .NET world.

What Are the Standard Layers in a .NET Web API?

Sub-question: *What layers do most ASP.NET Core projects use, and what is each one responsible for?*

Method: Library research using Microsoft Learn documentation and community best practices.

I started by reading Microsoft's own guidance on project structure for ASP.NET Core applications. The official docs are surprisingly opinionated about this. I also looked at several open-source .NET projects on GitHub to see how real teams organize their code. The goal was to find the common pattern, not an exotic one.

Why Separate Projects Instead of Folders?

Sub-question: *What's the actual difference between using folders inside one project versus creating separate class library projects?*

Method: Library research and available product analysis, comparing both approaches in practice.

This was a question I had personally. I'd already seen the folder approach (just making a Services/ and Repositories/ folder inside the API project), but my coach and other sources suggested separate projects. I wanted to understand what you actually gain from the extra effort. I looked at blog posts from experienced .NET developers and compared the two approaches side by side.

How Does Dependency Injection Connect the Layers?

Sub-question: *How does ASP.NET Core's dependency injection system wire the layers together at runtime?*

Method: Library research using Microsoft Learn and practical experimentation in the Grubs4Scrubs project.

I read the Microsoft Learn documentation on dependency injection in ASP.NET Core, then applied it directly. The docs explain the concept, but actually registering services in Program.cs and seeing them get injected into constructors is what made it click. I tested different service lifetimes (Scoped, Transient, Singleton) to understand when each one makes sense.

What Does Each Layer Look Like With ADO.NET?

Sub-question: *How do you implement the repository and service patterns when using raw ADO.NET instead of Entity Framework?*

Method: Library research and best good and bad practices analysis.

Most layered architecture examples online use Entity Framework, which I can't use for this project. So I had to adapt. I looked at examples of the repository pattern with raw SqlConnection and SqlCommand, and I studied what changes (and what stays the same) compared to the EF Core version. The interface contracts are identical, only the implementation differs.

Results: Sources Reviewed

The following sources contributed to answering the research questions. Each one provided a different angle on the same topic.

Microsoft Learn: ASP.NET Core Architecture

Microsoft's official documentation on clean architecture for ASP.NET Core applications was the primary source. It describes a layered approach with a Core (domain) project, an Infrastructure (data access) project, and a Web (API) project. The naming is different from what I used, but the principle is the same: dependencies flow inward, and the domain layer has no dependencies on anything else.

Microsoft Learn: Dependency Injection in ASP.NET Core

This doc covers how the built-in DI container works. It explains service lifetimes (Scoped creates one instance per HTTP request, Transient creates a new one every time, Singleton creates one for the entire app lifetime), how to register interfaces with implementations, and how constructor injection works. The key takeaway: you register `IRecipeRepository` with `RecipeRepository` in Program.cs, and ASP.NET handles the rest.

Steve Smith (Ardalis): Clean Architecture Template

Steve Smith maintains a popular open-source template for ASP.NET Core projects that follows clean architecture principles. His template uses separate projects for Core, Infrastructure, and Web. Looking at his structure helped me understand how real production apps organize their code. His approach enforces the rule that Infrastructure (database) code can't leak into the Web (controller) layer because the projects don't reference each other directly.

Microsoft Learn: Repository Pattern

The Microsoft docs describe the repository pattern as a way to abstract data access behind an interface. The controller doesn't know or care whether you're using SQL Server, PostgreSQL, or a text file. It just calls `IRecipeRepository.GetAll()` and gets recipes back. This separation makes testing easier and means switching databases later doesn't require changing business logic.

Andrew Lock: ASP.NET Core in Action

Lock's book covers project organization in depth. He recommends separate projects for anything beyond a small prototype, specifically because the compiler enforces the dependency rules. If your API project doesn't reference the DataAccess project, you literally can't write code that bypasses the service layer. It's not about discipline, it's about making the wrong thing impossible.

Conclusion: Sub-Questions

Each sub-question pointed toward the same general answer, but from a different direction. Here's what I found.

Standard Layers, Answered

Most ASP.NET Core projects use four layers: Domain (plain models), DataAccess or Infrastructure (database queries), Business or Services (logic and validation), and API or Web (controllers). The names vary between sources, but the responsibilities don't. Domain holds the shape of the data. DataAccess talks to the database. Business validates and orchestrates. API handles HTTP. Dependencies only point downward.

Separate Projects vs. Folders, Answered

Folders are fine for small projects. But separate class library projects give you compiler-enforced boundaries. If your API project doesn't have a reference to DataAccess, you can't accidentally import a repository in a controller. For a Fontys portfolio project, separate projects also look better in architecture documentation and prove you understand the principle, not just the pattern.

Dependency Injection, Answered

ASP.NET Core's DI container is what makes the layers work together without tight coupling. You register `builder.Services.AddScoped<IRecipeRepository, RecipeRepository>()` in Program.cs, and the framework handles creating and injecting instances. The controller asks for `IRecipeService` in its constructor, gets a `RecipeService`, which itself received an `IRecipeRepository` the same way. The layers never create each other directly.

ADO.NET Implementation, Answered

The repository pattern works exactly the same with ADO.NET as with EF Core. The interface is identical (`GetAll()`, `GetById()`, `Create()`, etc.). Only the implementation changes. Instead of calling `_context.Recipes.ToList()`, you write a SqlConnection, open it, run a SqlCommand, and map the SqlDataReader rows to model objects. The service layer doesn't know or care which approach the repository uses, which is the whole point.

Conclusion: Main Question

The best way to structure the Grubs4Scrubs backend is as four separate class library projects in a single solution: Grubs4Scrubs.Domain for models, Grubs4Scrubs.DataAccess for repositories and SQL queries, Grubs4Scrubs.Business for services and validation, and Grubs4Scrubs.API for controllers and startup configuration. Dependencies flow strictly downward, and ASP.NET Core's dependency injection wires everything together at runtime.

For Grubs4Scrubs specifically, this means the recipe controller went from a 70-line file with embedded SQL to a thin 30-line file that just calls a service. The SQL moved to RecipeRepository, the validation moved to RecipeService, and the models live in their own project. Adding new features (like user authentication or meal planning endpoints) now means adding a new repository, a new service, and a thin controller, without touching existing code. The architecture also makes it easy to write about in portfolio documentation, since each layer has a clear, explainable purpose.

Summary

Grubs4Scrubs was restructured from a single flat API project into four separate class library projects: Domain, DataAccess, Business, and API. Each layer has a single responsibility, and dependencies only flow downward. The repository pattern abstracts database access behind interfaces, the service layer handles business logic and validation, and the controllers stay thin. ASP.NET Core's dependency injection connects everything. This approach makes the codebase easier to extend, easier to test, and easier to document for the Fontys portfolio.

References

All references are formatted in APA 7th edition style.

Microsoft. (2024). *Common web application architectures.* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

Microsoft. (2024). *Dependency injection in ASP.NET Core.* Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection

Smith, S. (2024). *Clean architecture solution template.* GitHub. https://github.com/ardalis/CleanArchitecture

Microsoft. (2024). *Design the infrastructure persistence layer.* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design

Lock, A. (2023). *ASP.NET Core in Action* (3rd ed.). Manning Publications.